

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники**

Блок задач с Timus
Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа Р3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

Содержание

1067. Структура папок.....	3
Условие:	3
Решение:.....	4
О решении:	5
1628. Белые полосы	6
Условие:	6
Решение:.....	7
О решении:	9

1067. Структура папок

Условие:

Хакер Билл случайно потерял всю информацию с жесткого диска своего компьютера, и у него нет резервных копий его содержимого. Но он сожалеет не о потере самих файлов, а о потере очень понятной и удобной структуры папок, которую он создавал и сохранял в течение многих лет работы.

К счастью, у Билла есть несколько копий списков папок с его жесткого диска. С помощью этих списков он смог восстановить полные пути к некоторым папкам (например, «WINNT\SYSTEM32\CERTSRV\CERTCO~1\X86»). Он поместил их все в файл, записав каждый найденный путь в отдельную строку.

Напишите программу, которая восстановит структуру папок Билла и выведет ее в виде отформатированного дерева.

Исходные данные

Первая строка содержит целое число N – количество различных путей к папкам ($1 \leq N \leq 500$). Далее следуют N строк с путями к папкам. Каждый путь занимает одну строку и не содержит пробелов, в том числе, начальных и конечных. Длина каждого пути не превышает 80 символов. Каждый путь встречается в списке один раз и состоит из нескольких имен папок, разделенных обратной косой чертой («\»).

Имя каждой папки состоит из 1-8 заглавных букв, цифр или специальных символов из следующего списка: восклицательный знак, решетка, знак доллара, знак процента, амперсанд, апостроф, открывающаяся и закрывающаяся скобки, знак дефиса, собаки, циркумфлекс, подчеркивание, гравис, открывающаяся и закрывающаяся фигурная скобка и тильда («!#\$%&'()-@^_`{~»).

Результат

Выведите отформатированное дерево папок. Каждое имя папки должно быть выведено в отдельной строке, перед ним должно стоять несколько пробелов, указывающих на глубину этой папки в иерархии. Подпапки должны быть перечислены в лексикографическом порядке непосредственно после их родительской папки; перед их именем должно стоять на один пробел больше, чем перед именем их родительской папки. Папки верхнего уровня выводятся без пробелов и также должны быть перечислены в лексикографическом порядке.

Пример

исходные данные	результат
7 WINNT\SYSTEM32\CONFIG GAMES WINNT\DRIVERS HOME WIN\SOFT GAMES\DRIVERS WINNT\SYSTEM32\CERTSRV\CERTCO~1\X86	GAMES DRIVERS HOME WIN SOFT WINNT DRIVERS SYSTEM32 CERTSRV CERTCO~1 X86 CONFIG

Решение:

```
#include <iostream>
#include <map>
#include <string>

struct Node {
    std::map<std::string, Node*> children;
};

void create_new_folder(Node* curr, std::string folder_name) {
    if (!curr->children.contains(folder_name)) {
        curr->children[folder_name] = new Node();
    }
}

void print_tree(Node* node, int depth) {
    for (const auto& [name, child] : node->children) {
        std::cout << std::string(depth, ' ') << name << std::endl;
        print_tree(child, depth + 1);
    }
}

int main() {
    int folders_count;
    std::cin >> folders_count;

    Node* root = new Node();
    for (int i = 0; i < folders_count; i++) {
        std::string path;
        std::cin >> path;

        Node* curr = root;
        std::string folder_name;
        for (char ch : path) {
            if (ch == '\\') {
                create_new_folder(curr, folder_name);

                curr = curr->children[folder_name];
                folder_name.clear();
            } else {
                folder_name += ch;
            }
        }
        create_new_folder(curr, folder_name);
    }
    print_tree(root, 0);
}
```

О решении:

Идея:

Пути разбиваются по компонентам, и по ним строится дерево каталогов. У каждой вершины дерева есть своя мапа дочерних папок, поэтому одинаковые префиксы переиспользуются, что помогает избежать повторных выводов папок. Все папки благодаря мапе отсортированы лексикографически.

Сложность по времени:

$O(n \log k)$ – каждый из n символов пути обрабатывается один раз, а вставка и создание новой папки стоит $O(\log k)$

Сложность по памяти:

$O(n)$ – так как в дереве хранится по одной вершине на одну уникальную папку и по одному ребру на каждый уникальный переход между нодами

1628. Белые полосы

Условие:

У каждого неудачника в жизни бывают не только чёрные, но и белые полосы. Марсианин Вась-Вась отмечает в календаре, представляющем собой таблицу $m \times n$, те дни, когда ему ужасно не повезло. Если Вась-Васю не повезло в j -й день i -й недели, то он закрашивает ячейку таблицы (i, j) в чёрный цвет. Все незакрашенные ячейки в таблице имеют белый цвет.

Будем называть отрезками жизни прямоугольники размером $1 \times l$ либо $l \times 1$. Белыми полосами Вась-Вась считает все максимальные по включению белые отрезки таблицы. А сможете ли Вы определить, сколько всего белых полос было в жизни Вась-Вася?

Исходные данные

Первая строка содержит целые числа m, n, k — размеры календаря и количество неудачных дней в жизни Вась-Вася ($1 \leq m, n \leq 30000$; $0 \leq k \leq 60000$). В следующих k строках перечислены неудачные дни в виде пар (x_i, y_i) , где x_i — номер недели, к которой относится неудачный день, а y_i — номер дня в этой неделе ($1 \leq x_i \leq m$; $1 \leq y_i \leq n$). Описание каждого неудачного дня встречается только один раз.

Результат

Выведите число белых полос в жизни Вась-Вася.

Примеры

исходные данные	результат
3 5 4 1 1 1 5 2 2 3 3	8
5 1 2 2 1 3 1	2

Решение:

```
#include <algorithm>
#include <iostream>
#include <unordered_set>
#include <vector>

long long to_val(int week_idx, int day_idx, int days_count) {
    return 1LL * week_idx * (days_count + 1LL) + day_idx;
}

int main() {
    int weeks_count, days_count, bad_days_count;
    std::cin >> weeks_count >> days_count >> bad_days_count;

    std::vector<std::vector<int>> bad_rows(weeks_count + 1);
    std::vector<std::vector<int>> bad_cols(days_count + 1);

    for (int i = 0; i < bad_days_count; ++i) {
        int week_idx, day_idx;
        std::cin >> week_idx >> day_idx;

        bad_rows[week_idx].push_back(day_idx);
        bad_cols[day_idx].push_back(week_idx);
    }

    long long horizontal_segments = 0, vertical_segments = 0;
    long long row_single_count = 0, col_single_count = 0, intersected_single_count = 0;

    std::unordered_set<long long> single_in_rows;
    single_in_rows.reserve((size_t)bad_days_count + weeks_count + 10);

    for (int row = 1; row <= weeks_count; row++) {
        auto& cols = bad_rows[row];
        std::sort(cols.begin(), cols.end());

        int segment_start = 1;
        for (int bad_day : cols) {
            int segment_length = bad_day - segment_start;
            if (segment_length > 0) {
                horizontal_segments++;
            }
            if (segment_length == 1) {
                row_single_count++;
                single_in_rows.insert(to_val(row, segment_start, days_count));
            }
            segment_start = bad_day + 1;
        }

        int segment_length = days_count - segment_start + 1;
        if (segment_length > 0) {
            horizontal_segments++;
        }
    }
```

```

    if (segment_length == 1) {
        row_single_count++;
        single_in_rows.insert(to_val(row, segment_start, days_count));
    }
}

for (int col = 1; col <= days_count; col++) {
    auto& rows = bad_cols[col];
    std::sort(rows.begin(), rows.end());

    int segment_start = 1;
    for (int bad_week : rows) {
        int segment_length = bad_week - segment_start;
        if (segment_length > 0) {
            vertical_segments++;
        }
        if (segment_length == 1) {
            col_single_count++;
            if (single_in_rows.count(to_val(segment_start, col, days_count))) {
                intersected_single_count++;
            }
        }
        segment_start = bad_week + 1;
    }

    int segment_length = weeks_count - segment_start + 1;
    if (segment_length > 0) {
        vertical_segments++;
    }
    if (segment_length == 1) {
        col_single_count++;
        if (single_in_rows.count(to_val(segment_start, col, days_count))) {
            intersected_single_count++;
        }
    }
}

std::cout
    << (horizontal_segments + vertical_segments - row_single_count -
col_single_count +
        intersected_single_count);
}

```


О решении:

Идея:

Черные клетки группируются отдельно по строкам и по столбцам. Для каждой строки после сортировки черных позиций считаются все белые горизонтальные отрезки между ними и у границ. Аналогично для каждого столбца считаются все белые вертикальные отрезки. Отрезки длины 1 нельзя сразу считать ответом, потому что такая клетка может входить в более длинную полосу в другом направлении, поэтому одиночные горизонтальные и одиночные вертикальные отрезки учитываются отдельно. В ответ берутся все горизонтальные и вертикальные отрезки, затем вычитаются все одиночные, и добавляются обратно только те клетки, которые были одиночными и по строке, и по столбцу, то есть действительно образуют максимальную полосу 1 на 1

Сложность по времени:

$O(k \log k + n + m)$ – так как каждая черная клетка один раз попадает в список своей строки и один раз в список своего столбца. Затем все эти списки сортируются за $O(k \log k)$. После сортировки все проходы по строкам и столбцам линейны, то есть $+ O(k + m + n)$

Сложность по памяти:

$O(k + m + n)$ – так как храним списки черных клеток по строкам и по столбцам, а также множество одиночных клеток по строкам